University of Camerino

School of Science and Technologies

Master Degree in Computer Science (Class LM-18)

Course of Process Mining

Analyzing an hospital billing dataset using Process Mining algorithms

Student
Riccardo Peruzzi

Supervisor
Prof. Barbara Re

A.A. 2025/2026

Contents

1 Introduction **7**

2 Log **9**
2.1 Reasons for the choice 9
2.2 General features 9
2.3 Log structure 10

3 Project **11**
3.1 Alpha algorithm 11
3.2 Heuristic algorithm 11
3.3 Tool : PM4PY 12

4 Results **13**
4.1 Analysis with Alpha algorithm 13
4.2 Analysis with Heuristic algorithm 13
4.3 Analysis with Heuristics algorithm (noise filtering) 14
4.4 Comparison table 17

5 Conclusion **19**

List of Figures

4.1 Alpha miner 14
4.2 Heuristic miner 14
4.3 Heuristic miner net 15
4.4 Heuristic miner with threshold 15
4.5 Heuristic miner net with threshold 16
4.6 Graphical results 18

1. Introduction

This paper describes a Process Mining project applied to the Hospital Billing Event Log dataset, a real log extracted from the ERP system of a regional hospital. The objective of the project is to compare the performance of two process discovery algorithms applied to the same log, in order to determine which approach produces the most accurate, understandable, and generalizable model of the real process. For the development of the project, the open-source library PM4PY was used, which offers established tools for loading and analyzing event logs, applying discovery algorithms, and calculating quantitative evaluation metrics.
The document is structured as follows. The first chapter describes the dataset used, illustrating its general characteristics, the log structure, and the motivations that guided its choice. The second chapter presents the algorithms employed and the tool used for their implementation. The third chapter reports the results obtained: for each algorithm, the discovered Petri net is shown. The chapter concludes with a comparative table of evaluation metrics and an interpretative analysis of the results, aimed at highlighting the strengths and limitations of each approach.

2. Log

The Hospital Billing Event Log dataset [1], made available through the `4TU.Research Data platform` , was used to carry out the project. The dataset represents a real-world example of an event log in healthcare and is widely used in the Process Mining literature for process discovery, conformity checking, and predictive analytics tasks. The log was extracted from the financial forms of a regional hospital's ERP system and contains information relating to the billing process for medical services provided. Each log trace represents the administrative management and billing cycle associated with a set of healthcare services provided to a patient. It is important to note that the dataset does not contain detailed clinical information on medical care provided, but only data relating to administrative and billing processes.

2.1 Reasons for the choice

The choice of the Hospital Billing dataset is primarily motivated by its real-world nature: it is an event log derived from an actual hospital process, not artificially generated. This aspect makes it possible to apply Process Mining techniques in a realistic, non-simulated scenario, working with data that reflect authentic operational dynamics and typical issues of administrative processes in healthcare.
A further element of interest is the significant size of the log. With approximately 100,000 cases, the dataset enables robust analyses and the observation of a wide variety of behaviors, including the presence of numerous process variants. The hospital billing process is also sufficiently complex for reasons that will be detailed later to make its discovery non-trivial and to justify the use of advanced algorithms.
Finally, the dataset is widely recognized and used within the Process Mining scientific community, a characteristic that attests to its reliability, validity, and relevance in research and experimentation.

2.2 General features

The dataset has considerable size, making it particularly suitable for applying Process Mining techniques in realistic scenarios. The log contains a large number of cases, each representing an administrative practice related to the hospital billing process. The recorded events describe the activities performed during the billing management cycle and cover a time period of approximately three years, between 2012 and 2016. The dataset includes numerous attributes associated with both cases and events, allowing not only the analysis of the process flow but also the study of contextual and organizational aspects. All data have been fully anonymized to ensure the privacy of the involved subjects: resource names, identifiers, and attribute values have been modified

or anonymized. Timestamps have also been randomized, while preserving the time intervals between events belonging to the same case. This approach preserves the temporal behavior of the process without compromising privacy.
Each case represents a specific instance of the administrative process, while events describe the individual activities performed during execution. The structure of the log follows the typical format of Process Mining: for each event, fundamental attributes are present, such as Case ID (unique identifier of the process), Activity (executed activity), Timestamp (moment of the event), Resource (resource that performed the activity), and Lifecycle Transition (state of the activity).
Among the activities present in the log, typical administrative events of the hospital billing process appear, which illustrate the variety and frequency of activities, reflecting the operational complexity of the healthcare domain, characterized by numerous administrative steps, checks, corrections, and interactions between different roles. This also reveals considerable heterogeneity in the paths followed by individual cases, subsequently justifying the adoption of discovery algorithms capable of handling flexible behaviors and noise.

2.3 Log structure

The preliminary analysis of the log provided a clear picture of the process structure. The log contains over 451,000 events, distributed across approximately 100,000 cases. Despite the large number of events, there are only 18 distinct activities, indicating that the process, although composed of a limited number of elementary steps, is repeated many times with very different frequencies. The predominant activity is `NEW`, with over 100,000 occurrences, followed by `FIN`, `RELEASE`, and `CODE OK`, each with frequencies exceeding 65,000 events. Other activities such as `CHANGE DIAGN` (over 45,000 occurrences) and `DELETE` (about 8,000) suggest the frequent presence of corrections or modifications to the cases. In contrast, activities like `ZDBC_BEHAN` appear only once, indicating exceptional behaviors or background noise.
A relevant aspect concerns the start and end activities. The only start activity is `NEW`, which initiates all cases, confirming the existence of a well-defined and standardized entry point. There are 14 end activities, reflecting the variety of possible outcomes of the process: most cases end with `BILLED` (almost 64,000 occurrences), but a significant number conclude with `DELETE` (over 8,000), `NEW` (about 22,000, suggesting possible restarts), or `FIN`. There are also less frequent terminations such as `REJECT`, `STORNO`, or `CODE NOK`, which denote negative or anomalous outcomes.
Finally, the log features as many as 1,020 process variants, i.e., different sequences of activities observed in individual cases. This high number of variants, given only 18 activities, indicates that the real process is extremely flexible and non-linear, with many alternative paths, loops, and conditional choices. This complexity fully justifies the adoption of advanced discovery algorithms, capable of handling unstructured behaviors and noise.

3. Project

This chapter describes the process discovery activities and techniques carried out on the Hospital Billing dataset. The goal is to apply two different process discovery algorithms specifically, the Alpha Algorithm and the Heuristics Miner, using the PM4PY library. For each algorithm, a model (Petri net) will be generated, visualized, and evaluated using quantitative metrics such as fitness, precision, generalization, and simplicity. The sequence of operations includes: the application of the Alpha Miner, the application of the Heuristics Miner in its standard configuration, and finally the application of the Heuristics Miner with filtering thresholds for noise reduction. The obtained results will be compared to determine which approach best represents the actual process.

3.1 Alpha algorithm

The Alpha Algorithm is one of the first algorithms developed in the field of Process Mining for performing process discovery tasks. Its main goal is to automatically reconstruct a process model from an event log by analyzing the order in which activities are executed across the different cases present in the dataset. The algorithm identifies causality, parallelism, and dependency relationships among activities by observing the sequences of events recorded in the log. Based on this information, a Petri net representing the behavior of the analyzed process is generated. Despite its historical and theoretical importance, the Alpha Algorithm has some limitations, particularly in handling complex processes characterized by noise, short loops, or duplicate activities. Nevertheless, it provides a fundamental basis for understanding how process discovery techniques work and remains one of the most well-known and studied methods in Process Mining.

3.2 Heuristic algorithm

The Heuristics Miner is a process discovery algorithm designed to overcome some of the limitations of the Alpha Algorithm, particularly in handling real-world data characterized by noise, exceptions, and infrequent behaviors. Unlike the Alpha Algorithm, the Heuristics Miner does not rely exclusively on direct relationships between activities but uses statistical measures of frequency and dependency to identify the most significant process paths. This approach makes it possible to filter out less relevant behaviors and obtain more robust and understandable models even in the presence of complex and large-scale logs. The algorithm is particularly well-suited to real-world scenarios where the process may exhibit numerous variants and operational deviations.

3.3 Tool : PM4PY

For the implementation of the Process Mining activities, the PM4Py (Process Mining for Python) library [3] was used, one of the main open-source libraries dedicated to process analysis using Python. PM4Py provides a comprehensive set of tools for loading, manipulating, and analyzing event logs, supporting numerous algorithms for process discovery, conformance checking, and performance analysis. The library allows working with standard formats such as XES and CSV and is widely used in both academic and industrial settings. Among the features offered are the application of algorithms such as Alpha Miner, Heuristic Miner, and Inductive Miner, the visualization of process models through Petri nets or BPMN, and the calculation of metrics related to process performance. The use of PM4Py made it possible to develop the entire project efficiently, leveraging established tools specifically designed for Process Mining.

4. Results

This chapter presents the models obtained from the application of the process discovery algorithms described above. For each algorithm, the resulting Petri net image is shown. Finally, quantitative evaluation metrics for each model are reported. The chapter concludes with a comparative analysis of the results obtained, highlighting the strengths and weaknesses of each approach. The source code is available in the Git Hub repository [2].

4.1 Analysis with Alpha algorithm

The Petri net discovered by the Alpha Miner (figure 4.1) reveals a structurally degenerate model. The process starts with the NEW activity and immediately fans out into a flat, parallel structure where the majority of activities including RELEASE, FIN, CODE OK, BILLED, REJECT, STORNO, DELETE, SET STATUS, MANUAL, EMPTY, JOIN-PAT, CHANGE DIAGN, and CODE NOK connect directly to a single final place with no intermediate ordering or dependencies between them. More critically, four activities ZDBC_BEHAN, CHANGE END, REOPEN, and CODE ERROR appear completely isolated from the rest of the net, with no incoming or outgoing arcs linking them to the main flow. This outcome is a well-known limitation of the Alpha Miner: the algorithm relies solely on direct succession relationships in the log and cannot handle implicit dependencies, short loops, or low-frequency behaviors, all of which are present in a real-world log like HospitalBilling.

4.2 Analysis with Heuristic algorithm

The Petri net produced (figure 4.2) by the Heuristics Miner presents a considerably more complex structure compared to the Alpha Miner model. The net contains a large number of places, transitions, and arcs, with multiple silent transitions distributed throughout the graph. The overall topology is dense and difficult to read visually, with overlapping paths running across the entire width of the model. Despite this complexity, a main flow can be identified running left to right from the initial place through the core activities of the billing process, with several branching and rejoining paths representing alternative and exception behaviors.
The heuristics net produced without filtering thresholds (figure 4.3) shows all 18 activities correctly placed in the graph, with weighted arcs indicating the frequency of transitions between them. The dominant flow is clearly visible: NEW initiates the process, leading into a dense cluster of interactions involving FIN, RELEASE, CODE OK, BILLED, and CHANGE DIAGN, which together account for the vast majority of observed events. Activities such as ZDBC_BEHAN (1 occurrence), CODE ERROR (75 occurrence),

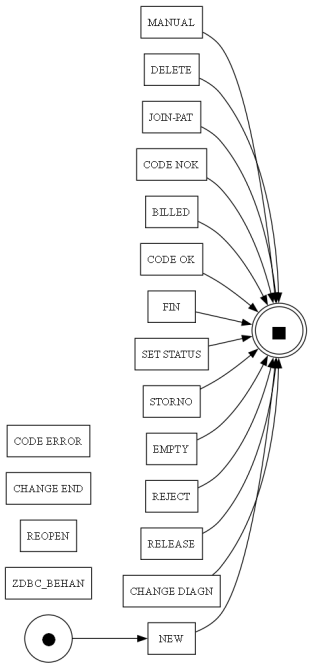

Figure 4.1: Alpha miner

and `CHANGE END` (38 occurrence) are also present, connected by low-weight arcs that reflect their marginal role in the process. The presence of these rare activities, while complete from a discovery standpoint, introduces noise into the model and contributes to the structural complexity observed in the corresponding Petri net.

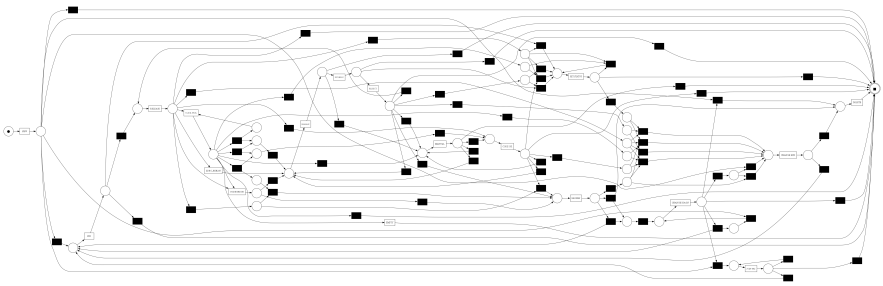

Figure 4.2: Heuristic miner

4.3 Analysis with Heuristics algorithm (noise filtering)

The Petri net derived from the filtered heuristic network (figure 4.4) is significantly more structured and readable than its unfiltered counterpart. The number of positions and transitions is small and the path of the main process proceeds more linearly from left to right, starting from `NEW` and passing through `CHANGE DIAGN`, `FIN`, `RELEASE` and in the `CODE OK` and `CODE NOK` branching. The sequence from `BILLED` to `STORNO` and subsequently to `REJECT` is clearly identifiable in the central portion of the network, while `REOPEN` and `DELETE` appear as distinct paths in the lower section. The overall topology is less cluttered and the flow between tasks is easier to track than the standard Heuristics Miner model.

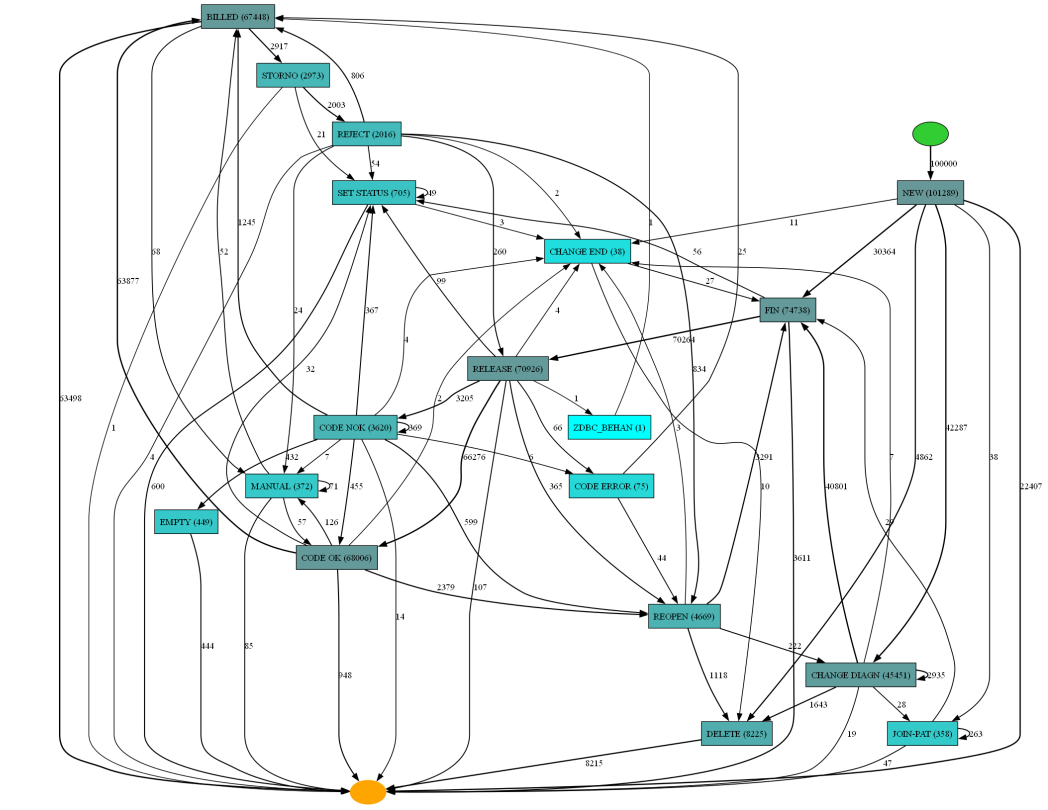

Figure 4.3: Heuristic miner net

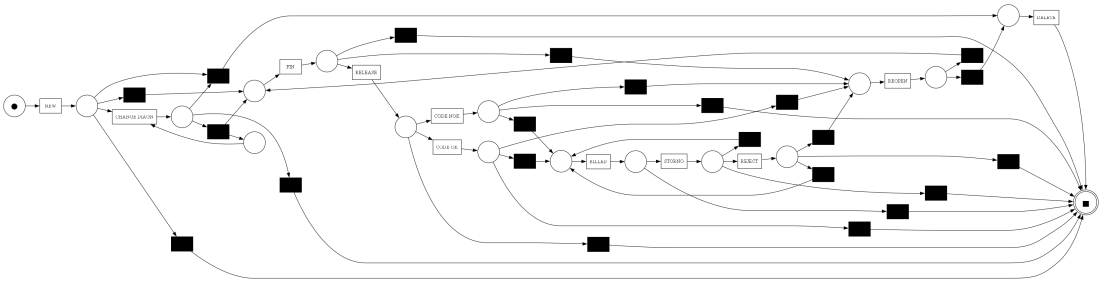

Figure 4.4: Heuristic miner with threshold

The filtered heuristic network (figure 4.5) has a substantially reduced and cleaner structure than the unfiltered version. The number of nodes is smaller, as several tasks no longer appear in the graph and the remaining edges connect a more focused set of nodes with higher frequency weights. The main flow follows a clear left-to-right progression from `NEW` through `CHANGE DIAGN` and `FIN`, continuing into `RELEASE` and then branching between `CODE OK` and `CODE NOK`, followed by the chain from `BILLED` to `STORNO` and finally to `REJECT`. `REOPEN` and `DELETE` are visible as distinct paths at the bottom of the graph. The overall layout is more readable and the relationships between tasks are easier to follow than the unfiltered heuristic network.

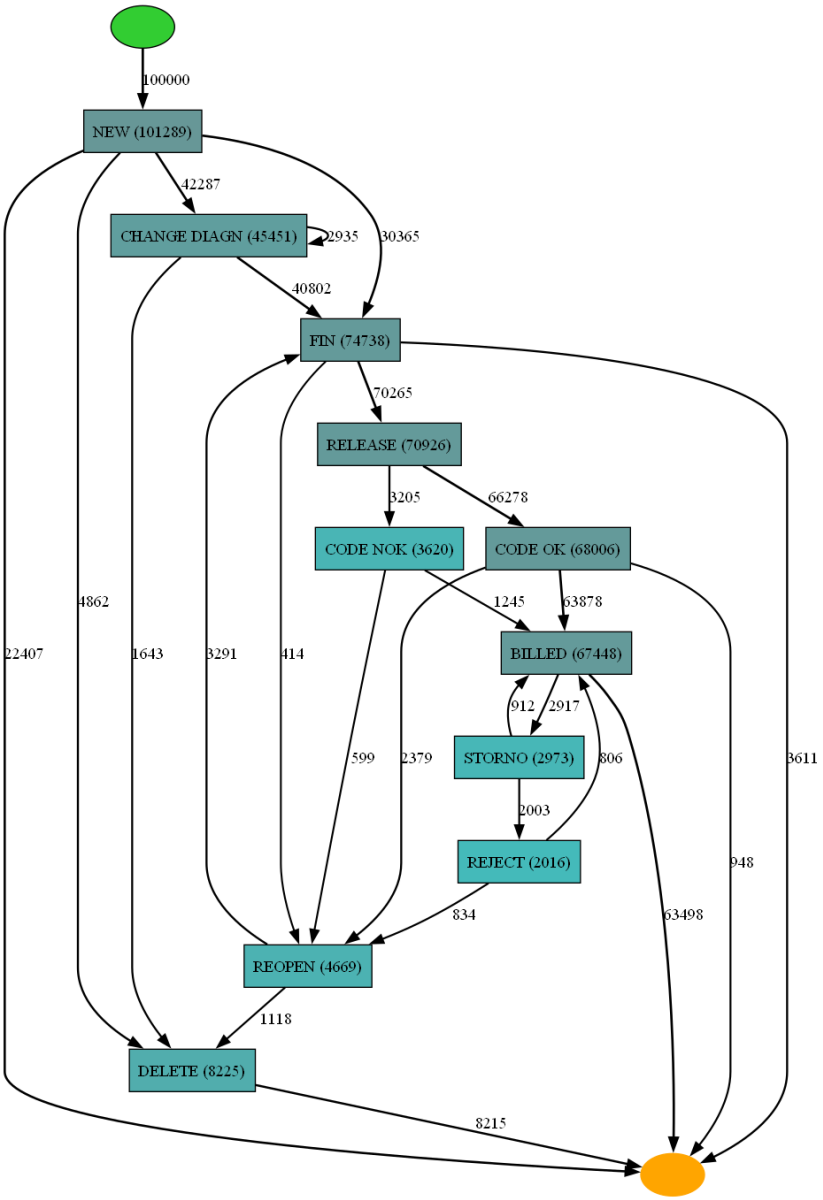

Figure 4.5: Heuristic miner net with threshold

4.4 Comparison table

In order to evaluate and compare the three obtained models, the comparative table 4.1 was constructed, which reports the values of the four evaluation metrics: fitness, precision, generalization, and simplicity. These were calculated for each of the algorithms used.

Metric	**Alpha Miner**	**Heuristics Miner**	**Heuristics (threshold)**
Fitness (log)	0.679755	0.917344	0.933242
Precision	0.386740	0.912390	0.997186
Generalization	0.912607	0.735766	0.934109
Simplicity	1.000000	0.472924	0.568182

Table 4.1: Comparison of performance metrics between Process Discovery algorithms

The Alpha Miner exhibits the lowest fitness (0.68), indicating that the model fails to reproduce about one third of the behaviors observed in the log. The precision is extremely low (0.39), meaning that the model allows many behaviors not present in the real log. These values are consistent with the degenerate structure of the Petri net observed earlier in Figure 4.1, characterized by flat parallelism and isolated activities. The generalization is instead high (0.91), but this is a misleading value, mainly due to the excessive simplicity of the model rather than a true ability to generalize. The simplicity is maximal (1.00), reflecting the extremely elementary structure of the net.
The standard Heuristics Miner drastically improves fitness (0.92) and precision (0.91), demonstrating that it captures the real process behavior much better. However, generalization drops to 0.74, signaling a risk of overfitting: the model adapts too faithfully to the specific log, losing the ability to predict future behaviors. The simplicity is low (0.47), consistently with the complex and dense Petri net obtained after converting the heuristics net, which is difficult to interpret visually.
The Heuristics Miner with thresholds represents the best compromise. Fitness rises further to 0.93, while precision reaches an excellent value of 0.997, indicating that the model admits almost exclusively behaviors that were actually observed. Generalization improves significantly (0.93), surpassing both the Alpha Miner and the standard Heuristics Miner: filtering out rare activities and edges has eliminated noise, making the model more robust and capable of generalizing. Finally, simplicity increases to 0.57, still low in absolute terms but higher than that of the standard Heuristics Miner, demonstrating that the application of thresholds has reduced the structural complexity of the model.

In summary, as is also possible seen in the figure 4.6, the Alpha Miner proves unsuitable for this real log due to its inability to handle noise and complexity. The standard Heuristics Miner produces a faithful but complex model prone to overfitting. The Heuristics Miner with filtering thresholds offers the best balance across all metrics, proving to be the most suitable approach for discovering a comprehensible, precise, and generalizable model of the hospital billing process.

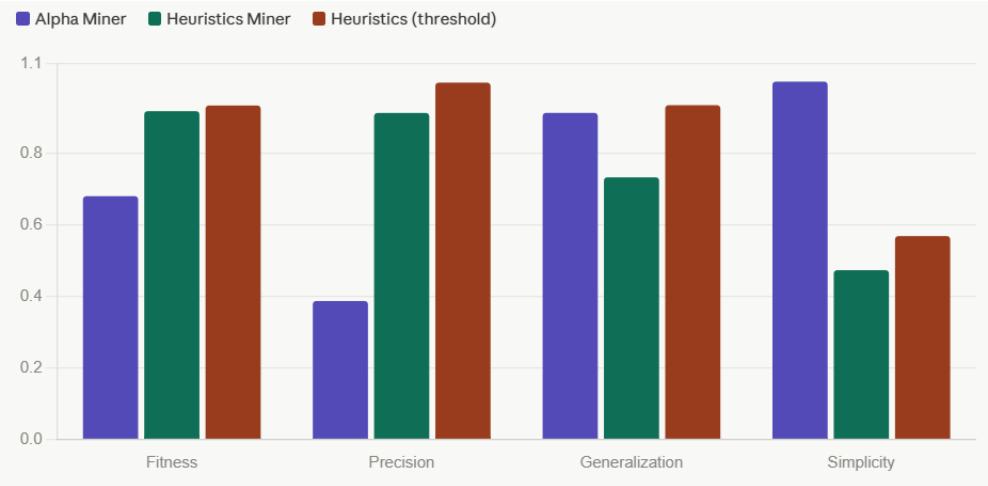

Figure 4.6: Graphical results

5. Conclusion

The project made it possible to compare three process models obtained by applying the Alpha Miner, the standard Heuristics Miner, and the Heuristics Miner with filtering thresholds to the Hospital Billing dataset. The results obtained clearly show how the choice of algorithm significantly influences the quality of the discovered model, especially in the presence of a real log characterized by noise, numerous variants, and infrequent behaviors.

The Alpha Miner proved inadequate for this type of data: despite achieving maximum structural simplicity, it produces a degenerate model with very low fitness and precision, unable to correctly represent the dependencies between process activities. The standard Heuristics Miner significantly improved fitness and precision, but at the expense of simplicity and generalization, producing a dense and poorly interpretable Petri net. The Heuristics Miner with filtering thresholds, on the other hand, demonstrated the best compromise across all considered metrics: high fitness and precision, superior generalization compared to the other two models, and a more readable structure, obtained by eliminating rare activities and edges that introduced noise into the model.

In conclusion, the conducted experiment confirms that for complex real logs, adopting an algorithm capable of handling noise along with appropriate filtering parameters is crucial for obtaining meaningful and usable process models in concrete application contexts.

Bibliography

[1] *Hospital Billing - Event Log*. https://data.4tu.nl/articles/dataset/Hospital_Billing_-_Event_Log/12705113/1. 2017.

[2] Riccardo Peruzzi. *Process_Mining-project*. https://github.com/RiccardoPeruzzi/Process_Mining-project. 2026.

[3] "pm4py: Process mining for Python". In: (). URL: https://processintelligence.solutions/.